Part 1: CI/CD Pipeline Design

sync-service

Spring Boot • MongoDB • GCP VMs • Jenkins

Property	Value
Service	sync-service (Spring Boot)
Database	MongoDB
Infrastructure	GCP Virtual Machines
CI/CD Tool	Jenkins
Environments	qa → staging → prod
Document Version	1.0

1. Branching Strategy

1.1 Branch → Environment Mapping

Each long-lived branch maps directly to a deployment environment:

Branch	Environment	Trigger	Auto-Deploy
feature/*	None	PR opened → CI only	No
develop	QA	Merge to develop	Yes
staging	Staging	Merge to staging	Yes
main	Production	Tag + manual approval	No

1.2 Preventing Accidental Production Deployments

Preventing Accidental Prod Deployments

- `The main` branch is **protected** — no direct pushes, PRs only
- The Jenkins job requires a **manual approval step** before deployment
- Separate Jenkins credentials per environment (prod creds not accessible in qa jobs)
- Tag-based prod releases — only tagged commits deploy to prod:

2. Jenkins Pipeline

2.1 On Pull Request (CI Only — No Deploy)

Every opened or updated PR runs a lightweight CI pipeline to give fast feedback:

Stage	Tool / Command	Purpose
Checkout	git checkout	Clone the branch
Build	mvn clean package -DskipTests	Verify compilation
Unit Tests	mvn test	Run unit test suite
Code Analysis	mvn sonar:sonar	SonarQube quality gate
Security Scan	dependency-check --scan .	OWASP CVE scan

```
pipeline {
  stages {
    stage('Checkout')       { /* clone repo */ }
    stage('Build')          { sh 'mvn clean package -DskipTests' }
    stage('Unit Tests')     { sh 'mvn test' }
    stage('Code Analysis')  { sh 'mvn sonar:sonar' }
    stage('Security Scan')  { sh 'dependency-check --scan .' }
  }
  // No deployment on PR
}
```

2.2 On Merge (Full Pipeline)

Merges to develop, staging, or main run the full pipeline, including build, test, image push, and deploy:

```
pipeline {
  stages {
    stage('Checkout')         {}
    stage('Build')            { sh 'mvn clean package' }
    stage('Unit Tests')       { sh 'mvn test' }
    stage('Integration Tests'){ sh 'mvn verify' }
    stage('Docker Build')     { sh 'docker build -t
sync-service:${GIT_COMMIT} .' }
    stage('Push Image')       { sh 'docker push
gcr.io/project/sync-service:${GIT_COMMIT}' }
    stage('Deploy to ENV')    { /* env-specific deploy */ }
    stage('Smoke Tests')      { sh './scripts/smoke-test.sh' }
    stage('Notify')           { /* Slack/email */ }
  }
}
```

Prod-Specific Manual Gate:

```
stage('Approval') {
  when { branch 'main' }
  steps {
    input message: 'Deploy to PROD?',
          ok: 'Approve',
          submitter: 'senior-devs,release-managers'
  }
}
```

2.3 Rollback Strategy

If smoke tests fail after deployment, Jenkins automatically rolls back to the previous known-good image:

```
stage('Deploy') {
  steps {
    script {
      try {
        deploy(version: env.GIT_COMMIT)
        runSmokeTests()
      } catch (err) {
        echo "Deploy failed -- rolling back to ${PREVIOUS_VERSION}"
        deploy(version: env.PREVIOUS_VERSION)
        currentBuild.result = 'FAILURE'
        notifySlack("Rollback triggered on ${ENV}")
        error("Deployment failed")
      }
    }
  }
}
```

> **PREVIOUS_VERSION** is stored as a Jenkins parameter, updated only after a successful deployment.

3. Configuration Management

3.1 Environment-Specific Configs

Spring profiles are used to separate environment configuration. The base application.yml holds shared settings; environment files contain overrides only:

```
src/main/resources/
  application.yml           # shared base config
  application-qa.yml        # QA overrides
  application-staging.yml   # staging overrides
  application-prod.yml      # prod overrides (NO secrets here)
```

The active profile is set via JVM argument at startup on each GCP VM:

```
java -jar sync-service.jar --spring.profiles.active=prod
```

3.2 Secrets Handling

Secrets are never stored in Git. All credentials are managed in GCP Secret Manager.

Secret Type	Storage	Access Method
MongoDB credentials	GCP Secret Manager	VM service account at startup
API keys	GCP Secret Manager	VM service account at startup

4. Deployment Strategy

4.1 Strategy Comparison

Recommendation: **Rolling Deployment**

Strategy	Downtime	Complexity	Resource Cost	Rollback Speed
Recreate	Yes	Low	Low	Slow
Rolling ✓ Chosen	None	Medium	Low	Medium
Blue/Green	None	High	High (2x VMs)	Instant

4.2 Why Rolling Deployment

Rolling is the best fit here because:

- GCP VMs (not Kubernetes) — Blue/Green requires double the infrastructure cost
- MongoDB connections are stateful — Blue/Green risks connection pool conflicts during cutover
- Rolling gives zero downtime with simpler infrastructure.